Fruit Ripeness Detection System

Rory O Loughlin

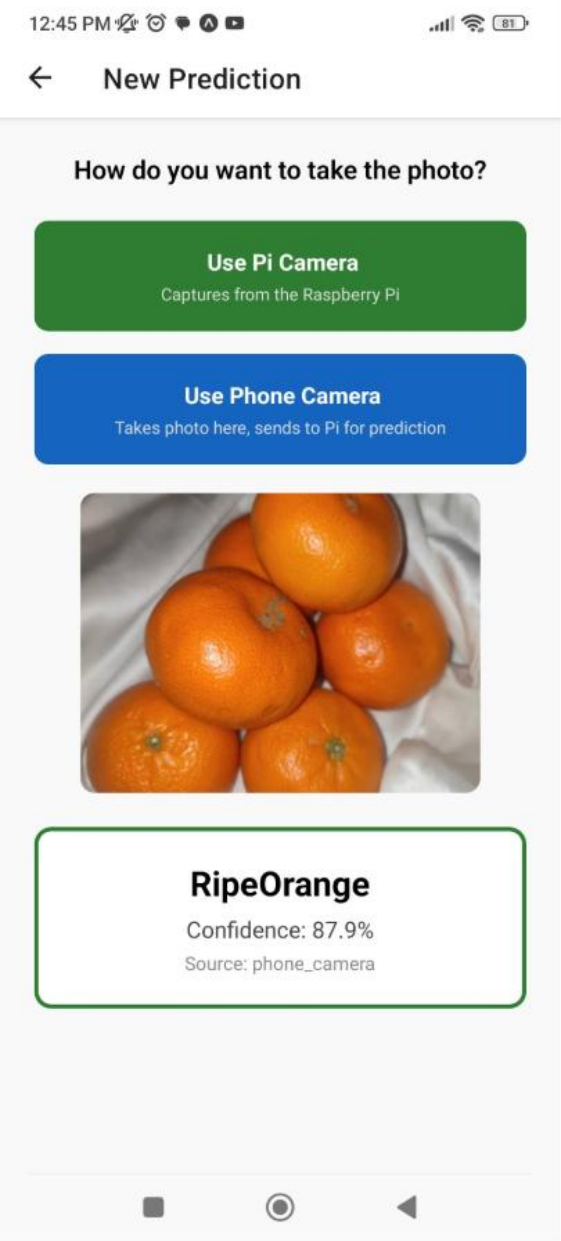
My project

Predict the level of ripeness of a piece of fruit through a user-friendly mobile app.

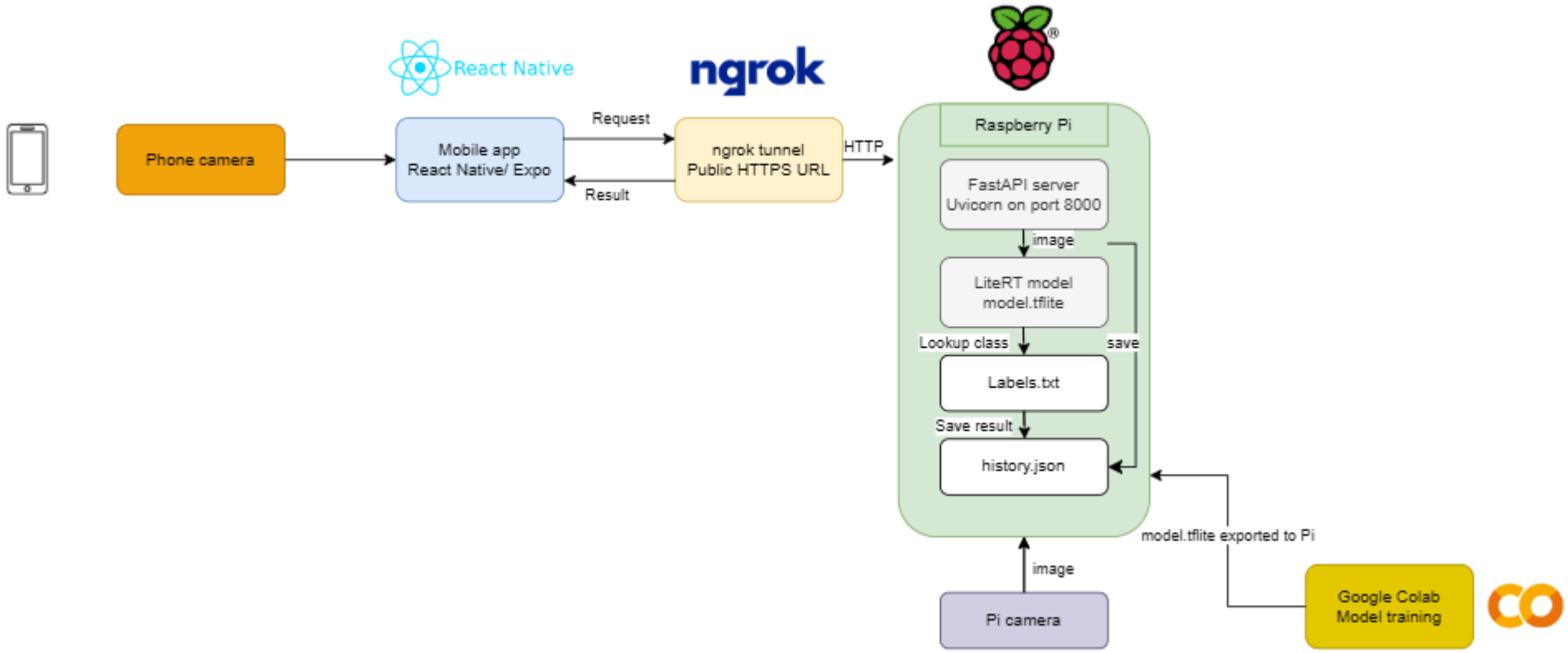
Combines a trained model and Raspberry Pi to detect ripeness via the app.

User can choose between Pi camera or Phone camera to capture an image.

Works for 2 fruits (banana, orange), 3 stages of ripeness (ripe, unripe, rotten)



System architecture



Key technologies used

Ngrok: Creates an HTTPS tunnel to connect the mobile app to the server on the Pi.

FastAPI & Uvicorn: Handles incoming requests from the app on port 8000.

LiteRT: Library used to run the TFLite model on the Pi. Replaced tflite-runtime because of compatibility issues with the Pi.

MobileNetV2: Pre-trained image classification model, used for model training.

Google Colab: Train and export the model.

Kaggle: Image dataset.

Training the model

- Used a fruit image dataset from Kaggle, 3 stages of ripeness, 6 classes in total, 800 images for each stage.
- 97.88% validation accuracy after 6 epochs.
- Built and trained a Python notebook using Google Colab.
- Knowledge from "Machine Learning with Python", also has a free GPU.
- Graphs and charts throughout the notebook to evaluate results (accuracy/loss curves, confusion matrix).
- Exported to Pi's filesystem using SCP.
- MobileNetV2: Used as a base model, I applied transfer learning by adding my own image classification layer on top.

The Raspberry Pi

- Contains the prediction API and the model.
- Connects to mobile app via Ngrok.
- All image capture and prediction logic runs here.
- Contains FastAPI server which runs the trained model using LiteRT, when called by the mobile app.
- Pi camera used as an alternative choice for predictions.
- History.json store the previous 20 predictions.

The mobile app

- Built with React Native & Expo.
- Displays project functionality.
- Home Screen: Entry point to navigate to capture or history screen.
- Capture Screen: Option to Capture an image using Pi camera or phone camera.
- History Screen: Displays previous 20 predictions, auto refreshes when screen comes into focus using useFocusEffect.

How a prediction works

- <u>Phone camera:</u>
- "Use Phone Camera" is pressed.
- Camera permission is requested using expo image picker.
- Photo is taken.
- App sends the image to the Pi.
- Pi receives the image, runs the trained model using LiteRT.
- Pi looks up the result in labels.txt, result gets saved to history.json.
- Returns JSON with label, score, source, timestamp.
- Result is displayed in the app.

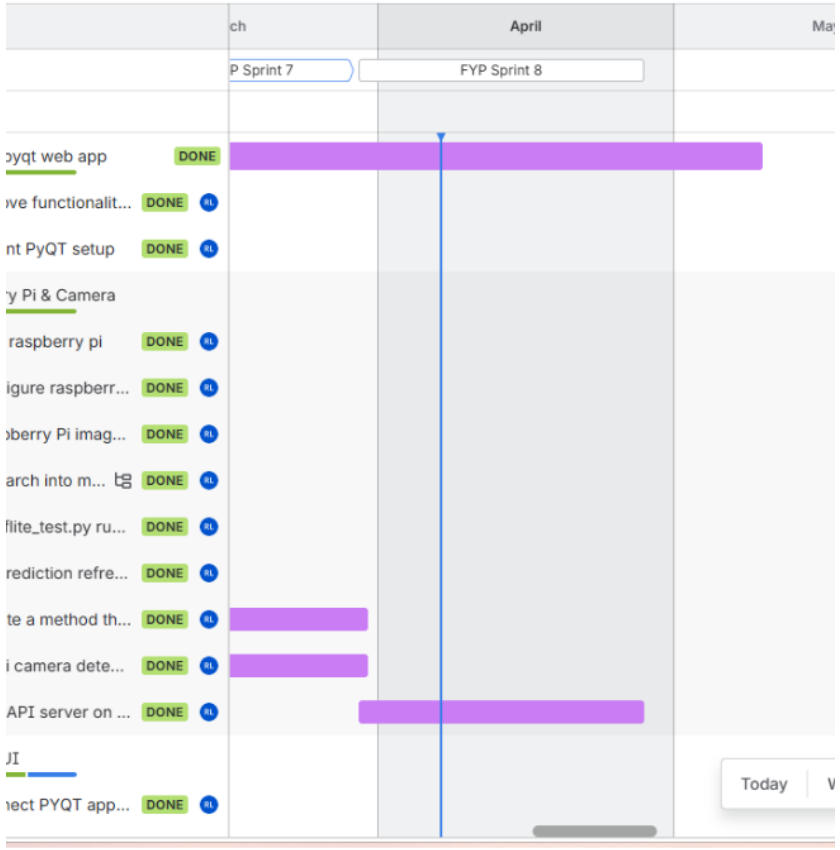
- <u>Pi camera:</u>
- "Use Pi Camera" is pressed.
- Mobile app calls GET/capture function on the Pi.
- Pi captures and image using the Pi camera.
- Trained model is ran using LiteRT, result is saved to history.json.
- Returns JSON with label, score, source, timestamp.
- Result is displayed in the app.

Problems faced

- Started off with web app, moved to mobile app for better mobile useability and access to the phone's camera.
- Getting the mobile app to talk to the Pi. URLs were hardcoded initially. Used Ngrok for a public HTTPS tunnel that will work on different networks.
- Tflite-runtime would not install correctly on the Pi's OS version, Switched to LiteRT which is supported.
- Image prediction inaccuracy: The images in the dataset are standardised with a consistent clean background and good lighting, therefore, these areas should be checked when capturing an image to achieve best results. Another cause of the inaccuracy was a model mismatch where the old model (7 output classes) was out of sync with the updated labels.txt (6 classes) which was fixed by replacing the old model with the updated one.

Project management

- Used Jira, OneNote and GitHub to log progress of the project.
- Weekly logs were created in OneNote and weekly standups were documented here along with AI prompts used in the project.
- Specific goals were put into tasks, bugs, and stories while Epics outlined broader tasks to be completed.
- Tasks were put into sprints which had a duration of 4 weeks.
- This helped me keep track of progress and the time consumption of tasks which allowed me to plan more efficiently.
- All progress completed was pushed to GitHub.



Testing

- 5 automated API tests send HTTP requests (status, history, predict, error handling, integration)
- Manual test cases for app navigation, camera permissions, predictions, error handling.
- Colab notebook tested with 80/20 train/validation split to check for overfitting, and out of distribution images to test on images it has not seen.

```
Running API tests...

PASS - /status returned online
PASS - /history returned a list with 20 entries
PASS - /predict returned label='UnripeOrange' score=0.5873
PASS - /predict correctly rejected empty request with 422
PASS - history saved entry with label='UnripeOrange' score=0.5873

Done.
```